

Object-Oriented Programming with JavaScript

Professional Course Guide

Course Overview

Welcome to the Object-Oriented Programming with JavaScript course.

This course is designed to teach students how to model real-world systems using objects and classes while developing maintainable, scalable, and reusable software solutions.

Object-Oriented Programming (OOP) is one of the most important paradigms in modern software engineering and serves as the foundation for enterprise applications, web development frameworks, game development, mobile applications, and large-scale software systems.

Throughout this course, students will learn how to analyze real-world entities, model them using classes and objects, apply the four pillars of OOP, and build professional applications using JavaScript.

By the end of this course, students will be able to:

- Understand object-oriented programming principles.
- Design software using classes and objects.
- Apply encapsulation, inheritance, polymorphism, and abstraction.
- Create reusable and maintainable code.
- Model real-world systems using OOP concepts.
- Develop medium-sized applications using JavaScript.
- Follow professional software design practices.
- Prepare for advanced software engineering topics.

Module 1 — Introduction to Object-Oriented Programming

Description

This module introduces the object-oriented paradigm and explains how it differs from procedural programming.

Learning Objectives

Students will be able to:

- Explain the purpose of OOP.
- Differentiate procedural and object-oriented programming.
- Identify objects in real-world systems.
- Understand the benefits of software modeling.

Topics Covered

Programming Paradigms

- Imperative Programming
- Procedural Programming
- Object-Oriented Programming
- Functional Programming

Why Object-Oriented Programming?

- Software complexity
- Reusability
- Maintainability
- Scalability

Real-World Modeling

- Objects around us
- Attributes and behaviors
- Software representation

Module 2 — Objects in JavaScript

Description

Students learn how JavaScript represents objects and how objects store data and behavior.

Learning Objectives

Students will be able to:

- Create objects.
- Access properties.
- Modify object data.

- Add behaviors to objects.

Topics Covered

Object Fundamentals

- Object literals
- Properties
- Methods

Working with Objects

- Reading properties
- Updating properties
- Deleting properties

Object References

- Reference types
 - Memory behavior
 - Object copying
-

Module 3 — Classes and Instances

Description

Students learn how classes serve as blueprints for creating objects.

Learning Objectives

Students will be able to:

- Create classes.
- Instantiate objects.
- Define attributes and methods.

Topics Covered

Classes

- Class syntax
- Blueprints
- Constructors

Instances

- Creating objects
- Instance lifecycle
- Multiple instances

Constructors

- constructor()
 - Initialization logic
 - Default values
-

Module 4 — Attributes and Methods

Description

Students learn how objects manage state and behavior.

Learning Objectives

Students will be able to:

- Create object attributes.
- Implement methods.
- Modify object state.

Topics Covered

Attributes

- Instance variables
- State representation

Methods

- Object behavior
- Invoking methods

State Management

- Updating data
 - Validation concepts
-

Module 5 — The this Keyword

Description

Students understand how object methods access their own data.

Learning Objectives

Students will be able to:

- Use this correctly.
- Understand execution context.
- Avoid common mistakes.

Topics Covered

Understanding this

- Object context
- Method invocation
- Context binding

Common Pitfalls

- Lost context
 - Arrow functions
 - Callbacks
-

Module 6 — Encapsulation

Description

Students learn how to protect data and control access to object internals.

Learning Objectives

Students will be able to:

- Hide implementation details.
- Create controlled interfaces.
- Protect object state.

Topics Covered

Encapsulation Principles

- Information hiding
- Controlled access

Accessors

- Getters
- Setters

Private Members

- Private fields
 - Data protection
-

Module 7 — Static Members

Description

Students learn the difference between instance-level and class-level behavior.

Learning Objectives

Students will be able to:

- Create static methods.
- Create static properties.
- Use utility classes.

Topics Covered

Static Methods

Static Properties

Utility Classes

Factory Methods

Module 8 — Inheritance

Description

Students learn how classes can reuse behavior from other classes.

Learning Objectives

Students will be able to:

- Implement inheritance.
- Reuse code effectively.
- Build class hierarchies.

Topics Covered

Inheritance Fundamentals

- Parent classes
- Child classes

extends Keyword

super Keyword

Method Reuse

Module 9 — Polymorphism

Description

Students learn how different objects can share common interfaces while exhibiting different behavior.

Learning Objectives

Students will be able to:

- Implement polymorphic behavior.
- Override methods.
- Design flexible systems.

Topics Covered

Method Overriding

Runtime Behavior

Polymorphic Design

Real-World Examples

Module 10 — Abstraction

Description

Students learn how to focus on essential behavior while hiding implementation complexity.

Learning Objectives

Students will be able to:

- Design abstract models.
- Simplify complex systems.
- Separate interfaces from implementation.

Topics Covered

Abstraction Concepts

Software Modeling

Abstract Design Patterns

Interface Thinking

Module 11 — Composition vs Inheritance

Description

Students learn modern software design techniques and when to favor composition.

Learning Objectives

Students will be able to:

- Compare composition and inheritance.
- Build flexible object structures.
- Avoid common design mistakes.

Topics Covered

HAS-A Relationships

IS-A Relationships

Composition

Aggregation

Design Trade-Offs

Module 12 — Object Relationships

Description

Students learn how objects collaborate within software systems.

Learning Objectives

Students will be able to:

- Design object interactions.
- Build interconnected models.
- Manage dependencies.

Topics Covered

Association

Aggregation

Composition

Dependency Relationships

Module 13 — UML Fundamentals for OOP

Description

Students learn how software engineers design systems before implementation.

Learning Objectives

Students will be able to:

- Read UML diagrams.
- Create class diagrams.
- Model software systems.

Topics Covered

UML Introduction

Class Diagrams

Attributes and Methods

Relationships

Visibility Modifiers

Module 14 — SOLID Principles Introduction

Description

Students are introduced to professional software engineering principles.

Learning Objectives

Students will be able to:

- Understand SOLID principles.
- Identify poor designs.
- Improve maintainability.

Topics Covered

Single Responsibility Principle

Open/Closed Principle

Liskov Substitution Principle

Interface Segregation Principle

Dependency Inversion Principle

Module 15 — Error Handling in OOP

Description

Students learn how objects manage exceptional situations.

Learning Objectives

Students will be able to:

- Handle errors professionally.
- Create custom exceptions.
- Improve application reliability.

Topics Covered

try / catch

throw

Custom Error Classes

Validation Strategies

Module 16 — Design Patterns Fundamentals

Description

Students are introduced to common object-oriented design patterns.

Learning Objectives

Students will be able to:

- Understand software patterns.
- Recognize reusable solutions.
- Apply simple patterns.

Topics Covered

Singleton Pattern

Factory Pattern

Builder Pattern

Observer Pattern

Module 17 — OOP Best Practices

Description

Students learn professional coding standards for object-oriented systems.

Learning Objectives

Students will be able to:

- Write maintainable code.
- Create readable architectures.
- Refactor object-oriented systems.

Topics Covered

Clean Code

Naming Conventions

Cohesion

Coupling

Refactoring Techniques

Module 18 — Object-Oriented Projects

Description

Students apply all learned concepts through complete software projects.

Suggested Projects

Beginner Projects

- Library Management System
- Vehicle Management System
- Employee Management System
- Banking Account Simulator

Intermediate Projects

- Hospital Management System
- E-Commerce Domain Model
- University Enrollment System
- Hotel Reservation System

Advanced Projects

- School Management Platform
- Inventory and Sales System
- CRM Simulation
- Airline Reservation System

Final Learning Outcomes

Upon successful completion of this course, students will be able to:

- Design software using object-oriented principles.
 - Model real-world systems through classes and objects.
 - Apply encapsulation, inheritance, polymorphism, and abstraction.
 - Build maintainable and scalable applications.
 - Create UML class diagrams.
 - Apply introductory SOLID principles.
 - Understand foundational software design patterns.
 - Develop professional object-oriented software solutions.
-

Recommended Learning Methodology

To succeed in this course, students are encouraged to:

1. Practice coding every day.
2. Model real-world objects before coding.
3. Draw UML diagrams before implementation.
4. Build projects consistently.
5. Refactor code regularly.
6. Analyze existing software designs.
7. Participate actively in coding exercises.

Object-Oriented Programming is best learned through consistent practice, software modeling, and project-based development.